Design Patterns Lab
Lecture 6

> Observer Pattern

Maryam Skaik, May-2026

Introduction to BDP

- Behavioral Design Patterns focus on **communication between objects**
- Define how **responsibilities are distributed** among components
- Describe how objects **interact and send messages**
- Emphasize **system behavior over time**, not just structure
- Aim to build systems that are:
  - **Loosely coupled**
  - **Highly interactive**
- Core idea: not *what objects are*, but *how they behave together*

Why Behavioral Patterns Matter

- In real systems, complexity comes from **interactions**, not creation
- Poor communication design leads to:
  - Tight coupling
  - Hard-to-maintain code
- Behavioral patterns help:
  - Organize communication
  - Control dependencies
  - Improve system flexibility
- Essential for scalable backend systems

Problems Solved by BDP

- Reduce **tight coupling** between communicating objects
- Enable **dynamic relationships at runtime**
- Support **event-driven systems**
- Allow extending behavior **without modifying existing code**
- Encapsulate complex communication logic
- Improve maintainability and scalability

Observer Pattern

- One of the most important Behavioral Patterns
- Widely used in real-world systems
- Solves the problem of:
  - Notifying multiple components about changes
- Introduces decoupled communication
- Used in:
  - Notifications
  - Event systems
  - Real-time updates

Problem Scenario (Store Example)

- A store sells products
- A customer wants an item that is out of stock
- The item will be available later
- Problem: how to inform the customer when it arrives?

Inefficient Solutions

- Option 1: Customer keeps checking
  - Repeated visits
  - Wastes time and effort
- Option 2: Store notifies everyone
  - Sends irrelevant notifications
  - Annoys uninterested customers
- Both approaches are:
  - Inefficient
  - Not scalable
  - Poor user experience

What We Actually Need

- A subscription-based system
- Customers should:
  - Choose if they want notifications
  - Choose what they care about
- Store should:
  - Notify only interested customers
- Requirements:
  - Flexibility
  - Selective communication
  - Loose coupling

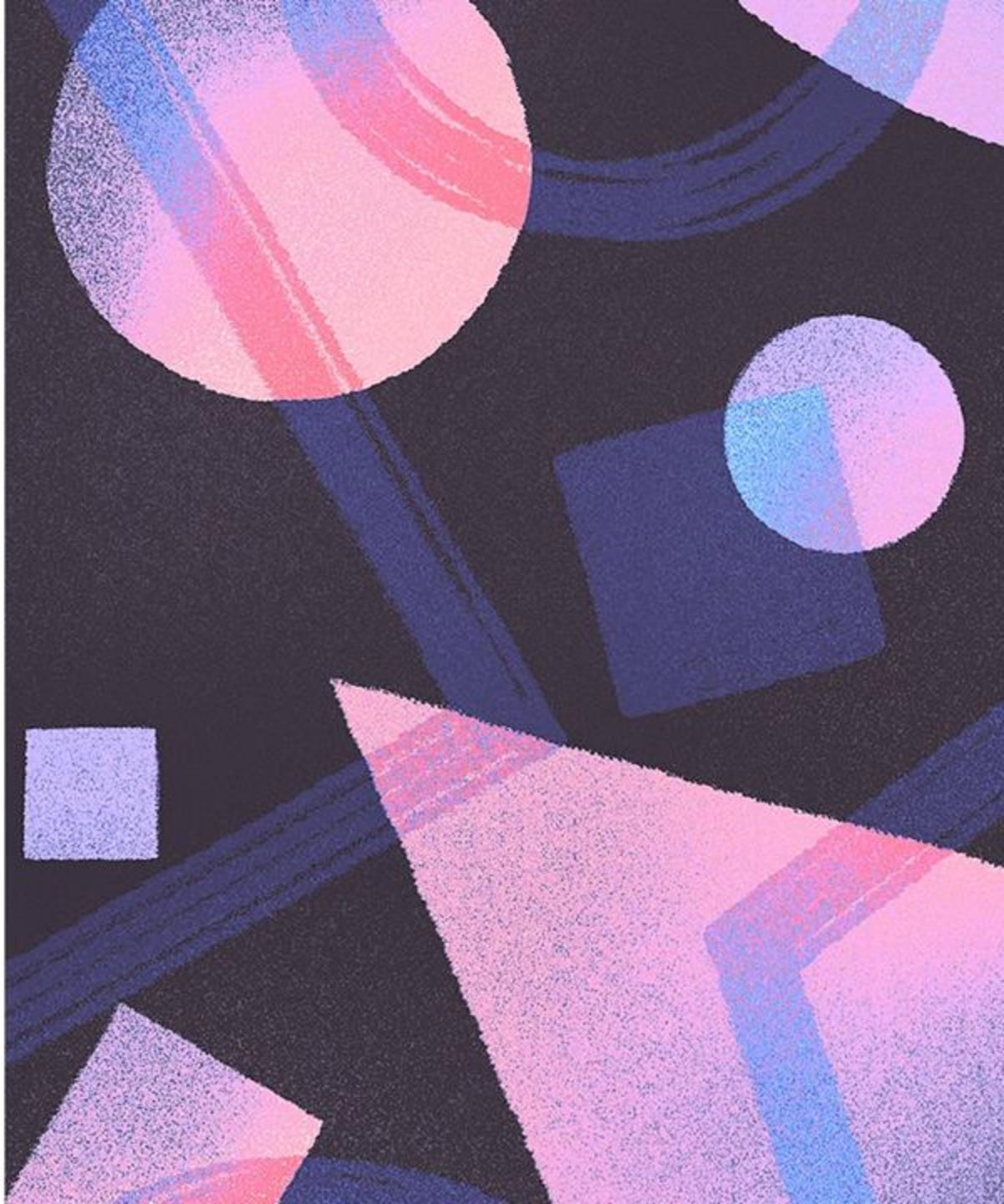
Observer Pattern Concept

- Defines a one-to-many relationship
- Components:
  - Subject (Publisher) → the source of changes
  - Observers (Subscribers) → interested listeners
- When state changes:
  - All subscribed observers are automatically notified
- Key idea:
  - “Don’t ask repeatedly — get notified when needed”

Benefits and Real-World Use

- Loose coupling between components
- Supports dynamic subscription at runtime
- Easy to extend:
  - Add new notification types without modifying existing code
- Follows **Open/Closed Principle**
- Common use cases:
  - Event-driven architectures
  - Notification systems
  - Messaging systems
  - Real-time applications (dashboards, chat apps)

Activity – Online Learning Platform

- System: online learning platform with **Courses** and **Students**
- Instructor adds: New lecture, Assignment
- Students: Enroll / unenroll in courses, Choose notification type (email, mobile)
- Requirement:
  - Notify only enrolled students when updates happen
- Task:
  - Design using Observer Pattern
  - Course → Publisher
  - Students → Subscribers

THANK YOU